

Universidad Autónoma de Occidente (UAO)

Facultad de Ingeniería — Sistemas Operativos

Taller de Kernel — 28 de Septiembre

Estudiante: Sergio Gómez Quiñones

1. Actividad de laboratorio: FreeBSD

1.1 Identificar el sistema

Estos tres comandos permiten identificar el sistema antes de compilar el kernel:

```
cat /etc/os-release
freebsd-version -u -k -r
getconf LONG_BIT
```

1.2 Configurar el kernel

```
cd /usr/src/sys/amd64/conf
mkdir /root/kernels
cp GENERIC /root/kernels/CLASEKERN
ln -s /root/kernels/CLASEKERN
```

1.3 Compilar e instalar

```
cd /usr/src
make buildkernel KERNCONF=CLASEKERN
make installkernel KERNCONF=CLASEKERN INSTKERNNAME=CLASEKERN
nextboot -k CLASEKERN
init 6
```

1.4 Verificación después de reiniciar

```
uname -a
freebsd-version -k -r
sysctl kern.conftxt | head
```

2. Trabajo investigativo: Tipos de Kernel

- Kernel monolítico (Linux): Todos los servicios corren en un solo espacio. Alto rendimiento, pero un fallo en un driver puede afectar el sistema.
- Kernel con diseño modular (FreeBSD): Carga y descarga módulos (.ko) en caliente sin recompilar.
- Sistema estructurado en capas (THE): Capas numeradas, fácil de depurar pero con costo de rendimiento por atravesar capas.
- Sistema con micronúcleo (MINIX 3): El kernel hace lo mínimo (IPC, procesos). Más robusto, menor rendimiento.
- Máquina virtual (IBM VM/370): Un monitor crea copias virtuales del hardware completo para aislar los SO.

2.1 Kernel monolítico: Linux

¿Qué es y de dónde viene? Linux es el kernel del sistema GNU/Linux. Lo empezó Linus Torvalds en 1991, cuando era estudiante en la Universidad de Helsinki, como un proyecto personal inspirado en UNIX y en MINIX. Lo publicó con licencia GNU GPL y desde entonces lo desarrollan miles de personas de forma colaborativa. Esas dos características aparecen en la diapositiva de características principales de la clase.

¿Cómo está organizado? Es un kernel monolítico: el planificador de procesos, la administración de memoria, el sistema de archivos virtual (VFS), los drivers y la pila de red están juntos en un solo programa que corre en modo kernel. Las aplicaciones corren en modo usuario y le piden cosas al kernel mediante llamadas al sistema (syscalls), como en el esquema de espacio de usuario y espacio de kernel de las diapositivas. Aunque es monolítico, también permite cargar módulos (LKM), así que en la práctica mezcla las dos ideas.

Lo que vi en la práctica: En el laboratorio compilé el kernel 6.18.54 en Ubuntu sobre WSL2. El resultado fue una imagen bzImage de 15 MB y 960 módulos .ko, y la compilación tardó casi 31 minutos con 12 núcleos. Eso me mostró que lo que va dentro de la imagen del kernel es solo una parte de todo lo que Linux soporta, y que el resto se compila como módulos.

Ventajas: Tiene muy buen rendimiento porque todos los componentes están en el mismo espacio y se llaman entre sí como funciones normales, sin pasar mensajes ni cambiar de contexto. Además es gratuito, tiene una comunidad enorme y soporta una cantidad muy grande de hardware y de arquitecturas.

Desventajas: Como todo corre con privilegios de kernel, un error en un solo driver puede provocar un kernel panic y detener todo el sistema. También es un código muy grande (la diapositiva menciona que el código fuente ocupa más de 400 MiB), lo que hace difícil mantenerlo y encontrar errores.

Dónde se usa: En servidores, en supercomputadoras, en sistemas embebidos y en Android, que usa un kernel Linux.

2.2 Kernel con diseño modular: FreeBSD

¿Qué es y de dónde viene? FreeBSD es un sistema operativo descendiente de BSD UNIX, que se desarrolló en la Universidad de California en Berkeley. Salió como proyecto propio a comienzos de los años 90 y se distribuye con licencia BSD. Se usa mucho en servidores y equipos de red.

¿Cómo está organizado? Tiene un kernel base y los controladores como módulos (.ko) que viven en /boot/kernel. Se pueden cargar y descargar con el sistema funcionando usando kldload y kldunload, y con kldstat se ve lo que hay cargado. También se puede cargar un módulo desde el arranque con una línea en loader.conf, por ejemplo if_ath_load="YES", como muestra la diapositiva del controlador inalámbrico ath. Es la misma idea que en Linux se llama LKM y se maneja con lsmod y modprobe.

Lo que vi en la práctica: Con este sistema hice la actividad principal de la clase. Creé la configuración CLASEKERN a partir de GENERIC, compilé el kernel (tardó 3924 segundos, más de una hora) y lo arranqué con nextboot. Después de reiniciar, uname -a ya mostraba

CLASEKERN y kldstat mostraba el kernel con un módulo cargado (intpm.ko). Algo que no esperaba es que en FreeBSD 15.1 el kernel viene instalado como paquete, así que make installkernel dio error y tuve que instalarlo en otra ruta con INSTKERNNAME.

Ventajas: Se puede agregar soporte para hardware sin recompilar todo el kernel ni reiniciar. Un kernel personalizado arranca más rápido y usa menos memoria que el GENERIC, como dice la diapositiva sobre el núcleo de FreeBSD.

Desventajas: Los módulos corren con los mismos privilegios que el kernel, así que un módulo con errores puede igual dañar el sistema. Además, compilar un kernel propio es lento y requiere tener las fuentes del sistema.

2.3 Sistema estructurado en capas: THE

¿Qué es y de dónde viene? THE fue un sistema operativo pequeño de procesamiento por lotes diseñado por Edsger Dijkstra y su equipo en la Technische Hogeschool Eindhoven, en los Países Bajos, y publicado en 1968. Es famoso porque fue el primer ejemplo claro de un sistema construido en capas, y porque ahí Dijkstra usó los semáforos para sincronizar procesos.

¿Cómo está organizado? El sistema se divide en capas numeradas y cada capa solo puede usar los servicios de la capa que tiene debajo. La capa 0 se encarga de la asignación del procesador y de la multiprogramación; la capa 1 de la administración de memoria; la capa 2 de la comunicación entre cada proceso y la consola del operador; la capa 3 de la entrada y salida; la capa 4 son los programas de usuario; y la capa 5 es el operador del sistema.

Ventajas: Como cada capa depende solo de la anterior, se puede construir y probar una capa a la vez. Si algo falla, es más fácil saber en qué nivel está el problema, y eso hace más sencillo depurar y verificar el sistema.

Desventajas: Es difícil decidir qué función va en cada capa, porque una capa solo puede usar las de abajo. Además, una petición tiene que atravesar varias capas y eso le quita rendimiento frente a un kernel monolítico.

Dónde se ve la idea hoy: THE casi no se usa como sistema real, pero la idea de capas sigue presente, por ejemplo en los modelos de capas de las redes y en la forma de organizar partes de otros sistemas operativos.

2.4 Sistema con micronúcleo: MINIX 3

¿Qué es y de dónde viene? MINIX lo creó Andrew Tanenbaum en la Vrije Universiteit de Ámsterdam. La primera versión salió en 1987 y se hizo para enseñar sistemas operativos, y de hecho fue una de las inspiraciones de Linux. MINIX 3 es la versión nueva, pensada para que el sistema sea muy confiable y se recupere solo de fallas.

¿Cómo está organizado? El micronúcleo hace lo mínimo: planifica procesos, maneja las interrupciones y permite que los procesos se comuniquen enviándose mensajes (IPC). Todo lo demás, como el sistema de archivos, la administración de procesos y los drivers, corre como procesos separados en modo usuario. Si un driver falla, un proceso llamado servidor de reencarnación lo detecta y lo vuelve a iniciar sin que se caiga el sistema completo.

Ventajas: Es más robusto y seguro, porque una falla en un driver queda aislada fuera del kernel y no lo tumba. El kernel es muy pequeño, así que hay menos código con privilegios y es más fácil revisarlo.

Desventajas: Pasar mensajes entre procesos cuesta más que llamar a una función dentro de un kernel monolítico, por eso suele tener menor rendimiento. Esta diferencia fue el centro de una discusión muy conocida entre Tanenbaum y Linus Torvalds en 1992 sobre qué diseño era mejor.

2.5 Máquina virtual: IBM VM/370

¿Qué es y de dónde viene? VM/370 es un sistema que IBM lanzó en 1972 para sus computadores centrales (mainframes) System/370. Venía de sistemas anteriores de la misma empresa como CP-40 y CP-67, y su descendiente actual es z/VM, que se sigue usando en los mainframes de IBM.

¿Cómo está organizado? Sobre el hardware real corre un programa de control (CP) que actúa como monitor de máquinas virtuales. Este programa crea varias máquinas virtuales que replican el hardware completo, y cada usuario ejecuta encima su propio sistema operativo. Lo más común era usar CMS, un sistema sencillo para un solo usuario. Así, cada usuario siente que tiene un computador completo para él solo, aunque todos comparten el mismo equipo físico.

Relación con lo que usé: La idea es la misma que usé en el laboratorio. La VM de FreeBSD corrió en VirtualBox sobre Windows 11, y WSL2 también ejecuta el kernel de Linux dentro de una máquina virtual ligera. Es decir, cada prueba de este taller se hizo sobre un sistema operativo ejecutándose dentro de una máquina virtual.

Ventajas: Aísla completamente a los usuarios entre sí y permite ejecutar varios sistemas operativos al mismo tiempo en el mismo equipo. También sirve para probar sistemas sin riesgo de dañar el equipo real.

Desventajas: Agrega una capa extra entre el sistema y el hardware, y eso consume rendimiento. Además, virtualizar el hardware completo es complejo y exige que el procesador permita controlar las instrucciones privilegiadas.

3. Versión 6: Instalación y Clasificación

3.1 Instalar un nuevo kernel en Ubuntu (Repositorios)

```
uname -r
apt search linux-image-generic-hwe
sudo apt update
sudo apt install linux-generic-hwe-$(lsb_release -rs)
sudo reboot
```

3.2 Clasificación de SO

- BeOS, Mac OS X, newOS: Kernel Híbrido.
- Mach, MINIX: Micronúcleo.
- AdeOS, EROS, KeyKOS: Nanonúcleo.

- Opal, Mungi, BriX: SASOS (Single Address Space Operating System).
- MIT exokernel: Exonúcleo.

4. Evidencias de Ejecución (Windows 11 / WSL2 / VirtualBox)

4.1 Ubuntu 26.04 sobre WSL2 (Windows 11)

```
$ uname -r
6.18.33.2-microsoft-standard-WSL2

$ wget linux-6.18.54.tar.xz
2026-09-28 22:30:48 URL:https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.18.54.tar.xz
[154801608/154801608]
-rw-r--r-- 1 sergio_gomez sergio_gomez 148M Sep 25 09:47 linux-6.18.54.tar.xz
```

Figura 1. Versión del kernel antes de empezar y descarga del código fuente de la versión 6.18.54.

```
#
# configuration written to .config
#
```

Figura 2. Configuración del kernel con make olddefconfig.

```
real    30m50.256s
user    293m32.196s
sys     54m55.183s
```

Figura 3. Tiempo de la compilación (make -j12 bzImage modules).

```

$ make kernelrelease
6.18.54-microsoft-standard-WSL2

$ ls -lh arch/x86/boot/bzImage
-rw-r--r-- 1 sergio_gomez sergio_gomez 15M Sep 28 23:01 arch/x86/boot/bzImage

$ file arch/x86/boot/bzImage
arch/x86/boot/bzImage: Linux kernel x86 boot executable, bzImage, version 6.18.54-microsoft-
standard-WSL2 (sergio_gomez@DESKTOP-8T2JB98) #1 SMP PREEMPT_DYNAMIC Mon Sep 28 22:59:50
-05 2026

$ find . -name "*.ko" | wc -l
960

$ make modules_install INSTALL_MOD_PATH=~/.kernel/rootfs
rc=0
6.18.54-microsoft-standard-WSL2

$ modinfo ../kernel/net/nsh/nsh.ko
license:      GPL v2
description:  NSH protocol
author:       Jiri Benc <jbenc@redhat.com>
intree:       Y

```

Figura 4. Resultado de la compilación: imagen bzImage, 960 módulos .ko e instalación de los módulos.

```

$ lsb_release -rs
26.04
$ uname -r
6.18.33.2-microsoft-standard-WSL2
$ lsmod | head -5

```

Module	Size	Used by
intel_rapl_msr	16384	0
intel_rapl_common	40960	1 intel_rapl_msr
kvm_intel	368640	0
kvm	1003520	1 kvm_intel

Figura 5. Versión de Ubuntu, kernel en ejecución y módulos cargados (lsb_release, uname -r, lsmod).

```

$ apt-get update 2>&1 | tail -4
Get:23 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [445
kB]
Get:24 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [89.7
kB]
Fetched 4229 kB in 4s (1000 kB/s)
Reading package lists...
$ apt search linux-image-generic-hwe 2>/dev/null | head -8
Sorting...
Full Text Search...
linux-image-generic-hwe-24.04/resolute-updates 7.0.0-38.38 amd64
  Generic Linux kernel image (transitional package)
linux-image-generic-hwe-24.04-edge/resolute-updates 7.0.0-38.38 amd64
  Generic Linux kernel image (transitional package)

```

Figura 6. Actualización de los repositorios (apt update) y búsqueda de los kernels HWE (apt search).

```

$ apt-cache policy linux-generic-hwe-$(lsb_release -rs) | head -5
linux-generic-hwe-26.04:
  Installed: (none)
  Candidate: 7.0.0-38.38
  Version table:
    7.0.0-38.38 500
$ apt-get -s install linux-generic-hwe-$(lsb_release -rs) | head -14
Reading package lists...
Building dependency tree...
Reading state information...
Solving dependencies...
The following additional packages will be installed:
  3cpio amd64-microcode bpffcc-tools bpftool bpftrace dracut dracut-core
  dracut-install firmware-sof-signed grub-gfxpayload-lists grub-pc grub-pc-bin
  grub2-common hwdata ieee-data intel-microcode iucode-tool libbpfcc
  libclang-cpp21 libclang1-21 libdebuginfod-common libdebuginfod1t64 libdw1t64
  libefiboot1t64 libefivar1t64 libevdev2 libgudev-1.0-0 libimobiledevice-1.0-6
  libimobiledevice-glue-1.0-0 libnl-3-200 libnl-genl-3-200 libnuma1 libpci3
  libplist-2.0-4 libtraceevent1 libupower-glib3 libusb-1.0-0 libusbmuxd-2.0-7
  linux-base linux-firmware linux-firmware-amd-graphics
  linux-firmware-amd-misc linux-firmware-broadcom-wireless

```

Figura 7. Kernel candidato en los repositorios y simulación de la instalación (apt-get -s install).

4.2 FreeBSD 15.1 en VirtualBox (Windows 11)

```
# cat /etc/os-release
NAME=FreeBSD
VERSION="15.1-RELEASE"
VERSION_ID="15.1"
ID=freebsd
# freebsd-version -u -k -r
15.1-RELEASE
15.1-RELEASE
15.1-RELEASE
# getconf LONG_BIT
64
# uname -a
FreeBSD freebsd 15.1-RELEASE FreeBSD 15.1-RELEASE releng/15.1-n283562-96841ea08dcf GENERIC
amd64
```

Figura 8. Identificación del sistema (os-release, freebsd-version, getconf y uname).

```
# cd /usr/src/sys/amd64/conf
# mkdir /root/kerne1s
# cp GENERIC /root/kerne1s/CLASEKERN
# ln -s /root/kerne1s/CLASEKERN
lrwxr-xr-x  1 root wheel 23 Sep 28 23:16 CLASEKERN -> /root/kerne1s/CLASEKERN
# grep ident CLASEKERN
ident      CLASEKERN
```

Figura 9. Creación de la configuración CLASEKERN a partir de GENERIC.

```
# diff /usr/src/sys/amd64/conf/GENERIC /root/kerne1s/CLASEKERN
21c21
< ident      GENERIC
---
> ident      CLASEKERN
# ls -l /usr/src/sys/amd64/conf/CLASEKERN
lrwxr-xr-x  1 root wheel 23 Sep 28 23:16 /usr/src/sys/amd64/conf/CLASEKERN ->
/root/kerne1s/CLASEKERN
```

Figura 10. Diferencia entre GENERIC y CLASEKERN (solo cambia la línea ident) y enlace simbólico en conf.

```
>>> Kernel(s) CLASEKERN built in 3924 seconds, ncpu: 6, make -j6
3933.05 real    17756.49 user    4551.05 sys
```

Figura 11. Compilación del kernel con make buildkernel (3924 segundos).

```
ERROR: The kernel at /boot/kernel was installed from packages.  
A packaged kernel should never be updated using installkernel;  
this will cause the package database to become out of sync with  
the live system state. Either uninstall the packaged kernel,  
or install this kernel to a different path using INSTKERNNAME.
```

Figura 12. Error de `make installkernel` porque el kernel de FreeBSD 15.1 está instalado como paquete.

```
# make installkernel KERNCONF=CLASEKERN INSTKERNNAME=CLASEKERN  
>>> Install kernel(s) CLASEKERN completed in 221 seconds, ncpu: 6  
# ls -lh /boot/CLASEKERN/kernel  
-r--r--r-- 1 root wheel 28M Sep 29 00:21 /boot/CLASEKERN/kernel  
# nextboot -k CLASEKERN  
# init 6
```

Figura 13. Instalación en otra ruta con `INSTKERNNAME`, `nextboot` y reinicio.

```
# uname -a  
FreeBSD freebsd 15.1-RELEASE FreeBSD 15.1-RELEASE CLASEKERN amd64  
# uname -i  
CLASEKERN  
# sysctl kern.bootfile  
kern.bootfile: /boot/CLASEKERN/kernel  
# sysctl kern.ident  
kern.ident: CLASEKERN  
# kldstat | head -3  
Id Refs Address Size Name  
1 5 0xfffffffff8020000 22fb980 kernel  
2 1 0xfffffffff82c1800 3220 intpm.ko
```

Figura 14. Verificación después de reiniciar: el sistema arrancó con `CLASEKERN`.

```
# uname -a  
FreeBSD freebsd 15.1-RELEASE FreeBSD 15.1-RELEASE CLASEKERN amd64  
# freebsd-version -k -r  
15.1-RELEASE  
15.1-RELEASE  
# sysctl kern.conftxt | head  
kern.conftxt: options CONFIG_AUTOGENERATED  
ident CLASEKERN  
machine amd64  
cpu HAMMER  
makeoptions WITH_CTF=1  
makeoptions DEBUG=-g  
options IICHID_SAMPLING  
options HID_DEBUG  
options EVDEV_SUPPORT  
options XENHVM
```

Figura 15. Segundo arranque con `CLASEKERN`: `uname`, `freebsd-version` y configuración del kernel (`kern.conftxt`).

```
# sysctl kern.bootfile
kern.bootfile: /boot/CLASEKERN/kernel
# kldstat | head -3
Id Refs Address          Size Name
 1    5 0xffffffff80200000 22fb980 kernel
 2    1 0xffffffff82c18000   3220 intpm.ko
```

Figura 16. Archivo del kernel en uso y módulos cargados con CLASEKERN.

```
# uname -a
FreeBSD freebsd 15.1-RELEASE FreeBSD 15.1-RELEASE releng/15.1-n283562-96841ea08dcf GENERIC
amd64
# kldstat
Id Refs Address          Size Name
 1    5 0xffffffff80200000 2351bd0 kernel
 2    1 0xffffffff82c18000   3220 intpm.ko
 3    1 0xffffffff82c1c000   2178 smbus.ko
# kldload if_epair
# kldstat | grep epair
 4    1 0xffffffff82c1f000   539c if_epair.ko
# kldunload if_epair
# kldstat | grep -c epair
0
```

Figura 17. Módulos del kernel: kldstat, kldload y kldunload con el módulo if_epair.

```
# ls -ld /boot/kernel /boot/CLASEKERN
/boot/CLASEKERN
/boot/kernel
# ls -lh /boot/CLASEKERN | head -5
total 158 MB
-r--r--r--  1 root wheel  121K Sep 29 00:24 aac.ko
-r--r--r--  1 root wheel  119K Sep 29 00:24 aacraid.ko
-r--r--r--  1 root wheel   16K Sep 29 00:24 accf_data.ko
-r--r--r--  1 root wheel   17K Sep 29 00:24 accf_dns.ko
# ls /boot/kernel/*.ko | wc -l
858
```

Figura 18. Kernel instalado en /boot/CLASEKERN junto al kernel original en /boot/kernel.